

STOCKLANA

Programmable PreStocks on Solana

A complete product, architecture, security, deployment, and competition submission brief

Surface	Purpose
PreStocks	Exact-mint tokenized-equity execution
Play	Fully collateralized payoff markets
Vault / MAQUE	Account fabric and payments
Agent Network	Policy-scoped autonomous spending
System / Proof	Solvency, accounting, receipts
Documentation	Judge path and source map

Executive summary

Stocklana is a Solana-native financial workstation built around a simple product thesis: a tokenized equity should remain useful after a user buys it. Most tokenized-equity experiences stop at a chart and a swap button. Stocklana treats an eligible PreStock as a programmable financial object that can move through wallet execution, collateralized markets, payments, commerce, credit, portfolio automation, and bounded agent authority. The interface is intentionally consumer-facing, but the underlying system is designed to make every important boundary explicit: who signs, where collateral sits, which provider executed, what was observed, and which receipt proves completion.

The product in one sentence

Stocklana connects Phantom-owned Solana assets to a single account fabric where an eligible PreStock can be bought, automated, collateralized, played, sent, spent, or delegated to an AI under visible policy controls. The system is not a brokerage clone and it is not an unbacked protocol token. Its differentiator is the post-purchase utility layer: the asset becomes an input to other useful financial actions while the wallet remains the signing boundary and the ledger remains reconstructible.

The user experience and marketing wedge

The public shell leads with the line Your money should not stop working after you invest it. That line is not decoration; it is the product positioning. The home experience moves from a cinematic hero into six immediate actions: buy a company, convert money, play, shop, send, and give an AI a budget. A wallet card shows the state of SOL, USDC, and PreStocks. Featured assets, payoff games, commerce, agent budgets, and activity are presented as one connected economy. Internal surfaces retain the same neon-black, acid-lime, cyan, and violet grammar so the judge does not feel like they have left the product when they inspect infrastructure.

The judge path

A strong demonstration can be completed in roughly ninety seconds. First connect Phantom and show that the wallet is the signer, not a custodial account. Open PreStocks and select an exact eligible mint. Build a Jupiter Swap V2 action and show that the order is only a plan until the provider and wallet return evidence. Open Play and demonstrate the Solana judging loop: connect, create a market, stake collateral, resolve with a commitment, and redeem. Move to Vault or MAQUE to show internal transfers, payment requests, or a one-time spend policy. Open Agent Network to show bounded budgets and approvals. Finish in System or Proof where solvency, accounting, provider boundaries, and receipts are legible.

Why Solana is essential

Solana is important because the same asset can remain a wallet-addressed programmable token across multiple product contexts. The PreStocks lane uses exact SPL mint eligibility and wallet-signed Jupiter routes. The Stocklana market program uses USDC escrow, deterministic accounts, position issuance,

time-gated resolution, and proportional redemption. The application can therefore connect trading, games, payments, and credit without turning each surface into a disconnected database feature. Low transaction cost and fast confirmation are part of the user experience, not merely infrastructure claims. The browser prepares actions; Phantom signs them; Solana supplies the public settlement boundary.

Market factory and payoff products

Stocklana includes a market catalog that generates 360 instruments across 16 families from the eligible PreStocks universe. These include valuation thresholds, token-price thresholds, premium and discount convergence, absolute moves, relative returns, valuation spreads, joint-positive outcomes, basket leaders, universe leaders, Gain Games, Downside Shields, Margin Duels, and Green-Majority basket games. The important safety choice is that the default execution model is fully collateralized pari-mutuel accounting. Every stake enters segregated per-market escrow, fees move separately, the winning pool receives only available collateral, paidOut reduces remaining liability, and Stocklana does not take the opposing YES or NO position.

The Solana program

The on-chain program in programs/stocklana-market is the executable market-state layer. It enforces the exact eight-mint PreStocks allowlist, USDC collateral, market and position accounts, vault and fee authorities, collateral transfers into a program vault, separate fee accounting, YES and NO pools, transferable positions, time-gated resolution, and proportional redemption. Resolution stores a 32-byte commitment so an external observation proof can be connected to the on-chain outcome. The matching JavaScript client derives the same accounts and constructs initialize, buy, transfer, resolve, redeem, and fee-withdraw instructions. No invented program identity is presented as deployed; deployment must return a real program ID and transaction evidence.

Wallet, authentication, and authority

Phantom is both the user experience and the security boundary. Stocklana reads public balances, requests a one-time challenge, verifies the wallet signature, and issues a session whose authority is tied to the verified public key. Private keys never enter the API. Account-change events refresh the workspace when the user switches accounts. A market, payment, deposit, or commerce action is not authorized merely because a browser sends a trader field; the production path requires the authenticated session and, where applicable, an external transaction signature. This model lets the app be friendly without pretending that an API request is a wallet proof.

V2 financial substrate and accounting

The V2 substrate gives financial state explicit semantic types: SL-CASH, SL-ESCROW, SL-COLL, SL-FEE, SL-CREDIT, SL-AGENT, SL-POS, and SL-BASKET. Internal double-entry accounting is the source of truth for monetary conservation. Token-2022 is used selectively for programmable public receipts such as restricted escrow, collateral, credit, agent, position, and basket claims. SL-CASH does not become a casually issued bearer stablecoin. The accounting layer tracks segregated backing, locked collateral, funded credit,

fees, agent budgets, position units, and event idempotency. This separation is a core trust feature because it makes non-cash authority and cash claims distinguishable to users, agents, and auditors.

Vault, payments, and commerce

Stocklana Vault gives users CASH, TRADING, RESERVE, and related subaccounts that can be reconciled with external deposits and withdrawals. MAQUE provides handles, payment requests, QR and NFC-friendly intents, offline authorization packets, and later settlement receipts. Offline authorization is deliberately not mislabeled as offline blockchain finality. Commerce uses one-time merchant-bound policies with an amount, merchant or MCC restriction, expiry, and approval threshold. Funds are reserved before capture. External card processors are represented through opaque provider identifiers; PAN and CVV are not stored. The result is a human-friendly payment and commerce surface backed by explicit authorization and reconciliation rather than a generic pay button.

AI Agent Vaults

The agent system is designed for bounded autonomy. A scoped capability secret is returned once while only its commitment is retained. Each Agent Vault has separate account categories, budgets, permissions, approval thresholds, market permissions, reconciliation, billing metadata, positions, and connector descriptors. The same finance and commerce code handles both human and agent actions, which avoids building an un-auditable parallel economy. Sentinel controls cover replay, velocity, and anomaly signals. The product promise is not that an AI can freely control money; it is that an AI can operate inside a policy the human can inspect, revoke, and require to ask for approval.

Security and post-quantum architecture

Stocklana layers cryptography, authorization, accounting, and external verification instead of treating one as a replacement for the others. Every finance event is domain-separated and linked with a BLAKE2b-512 commitment. High-risk settlement events can receive hybrid ML-DSA-65 and Ed25519 signatures. Routine events are batch-anchored every sixteen events for throughput. Private journal records use AES-256-GCM. The journal root key is protected by a hybrid ML-KEM-768 and X25519 envelope. Withdrawal follows REQUEST, RESERVED, AUTHORIZED, CHAIN TRANSFER, RPC VERIFIED, SETTLED. External signatures are single-use settlement references. These controls are meaningful only when the deployment keeps keys and provider secrets out of the browser and release archive.

External-provider boundaries

The system integrates with Jupiter, Pyth, Kamino, Meteora, Pons, card providers, and bank or debit rails through explicit adapters. The code can build a transaction or quote without claiming that the world changed. A Jupiter plan is not a fill; a Kamino transaction builder is not a deposit; a Pons launch request is not a token until the chain confirms it; a card policy is not an issued card until the provider returns its receipt. The app uses status language and receipt surfaces to keep those distinctions visible. This is strategically important in a competition submission because it demonstrates production judgment instead of hiding missing credentials

behind simulated success.

Deployment model

Stocklana ships as two connected production layers. The Solana executable layer is deployed to Devnet for judging by default, with an explicit mainnet path available later. The browser layer is built into static files and can be uploaded permanently through Turbo and Arweave using a Solana wallet. The API origin is runtime-configurable so the permanent frontend can call an orchestration service while Phantom signs Solana actions locally. CORS must be configured to the exact frontend origin. The release runbook treats a deployment as complete only when the chain or provider returns a real receipt. The repository includes static build, verification, deployment, and release-preflight commands to make this boundary reproducible.

Competition strengths

The strongest competition story is coherence. The project is not one isolated smart contract and not one disconnected design mock. It demonstrates a product shell, a market factory, collateral accounting, an on-chain program, wallet-native execution, a payment fabric, commerce controls, agent policies, accounting tokens, post-quantum receipts, and a documented release boundary. The home surface makes the idea understandable in seconds; the deeper surfaces give technical judges evidence when they click further. The exact-mint allowlist, solvency checks, house exposure of zero, receipt rule, and explicit external-action boundary are especially defensible proof points.

Known limitations and honest status

The project should not claim production completion without live receipts. A configured program ID is not the same as a confirmed deployment. A static bundle is not the same as a public permanent URL. A provider adapter is not the same as a provider account with credentials. The current repository can be verified structurally and through local regression suites, but the submission should attach actual Devnet signatures, an Arweave manifest ID, verified CMESH state, and a short recorded demo if those are available. If any receipt is missing, label the surface source-ready or demo-ready rather than executed. This honesty strengthens the submission because the architecture already explains exactly where execution ends and evidence must begin.

Submission checklist

Before submitting, confirm that the GitHub repository is public and points to the final commit; the live site loads the same build as the repository; Phantom can connect on the intended network; the judge path is recorded without waiting on a provider that is not configured; the video shows one complete user journey rather than a tour of every menu; the PDF explains the product and the trust boundaries; and every external claim has a receipt or a clear status label. Include the repository URL, live URL, video URL, Solana program ID and transaction, permanent frontend manifest, and any provider evidence in the submission form. Keep the Documentation tab as the in-product index for all of this material.

Source appendix

The following repository documents are the source of truth for the claims summarized above. They are included as a navigable appendix so the PDF can stand alone during submission review.

README.md

Stocklana - Programmable PreStocks on Solana

****Your money shouldn't stop working after you invest it.****

Stocklana is a desktop-first web app where eligible PreStocks become programmable financial objects. A user can connect Phantom, buy a PreStock, convert SOL/USDC, enter fully collateralized payoff games, spend through governed commerce rails, send money, or give an AI a bounded budget - all from one interface.

The public experience is intentionally simple: ****PreStocks · Convert · Play · Shop · AI · Send****. The deeper market factory, credit, launch, accounting, settlement, proof, and deployment systems remain underneath the same application.

> ****CMESH is not a PreStock.**** CipherMesh (CMESH) is Stocklana's separate Pons / Robinhood Chain platform-token lane and has its own dedicated tab. It is never added to the PreStocks eligibility registry.

Approved visual system

The current product shell follows Stocklana's neon-black / acid-lime / cyan / violet design language: cinematic animated hero, action ribbon, wallet card, featured PreStocks, live game spotlight, commerce, AI budget, activity, and a distinct CMESH surface. The design is implemented in the canonical app; it is not a separate demo.

Deployment

- Solana program deployment: ``npm run deploy:program:solana``
- permanent static build: ``npm run build:static``
- permanent web upload authenticated by a Solana wallet: ``npm run deploy:frontend:solana``
- full release runbook: ``DEPLOY-SOLANA.md``

A deployment is only considered complete when the chain/provider returns a real receipt.

V2 financial token substrate

Stocklana now has a typed financial accounting layer underneath markets, Agent Vaults, MAQUE and tokenized-equity workflows. It is deliberately not an unbacked protocol coin.

****Internal token grammar:**** ``SL-CASH``, ``SL-ESCROW``, ``SL-COLL``, ``SL-FEE``, ``SL-CREDIT``, ``SL-AGENT``, ``SL-POS``, ``SL-BASKET``. Monetary claims are reconciled to segregated USDC or an already-funded pool; agent budgets and position units are explicitly marked non-cash. ``src/accounting_tokens.py`` compiles financial events into a balanced double-entry journal and exposes ``StocklanaFinancialDigest/v2`` to both humans and agents.

****V2 - Token-2022 programmable lane:**** ``src/token2022-accounting.js`` creates selected receipt classes with ``MetadataPointer``, ``NonTransferable``, or ``TransferHook`` extensions. Escrow/collateral/credit/agent receipts are restricted; position and basket receipts can carry transfer policy. ``SL-CASH`` stays internal rather than becoming a casually issued bearer stablecoin.

V2 is the product default. Internal accounting remains the source of truth while Token-2022 carries programmable public claims. See ``docs/V2-FINANCIAL-SUBSTRATE.md``.

Wallet-native money + internet commerce

Stocklana treats a connected Phantom wallet as a real funding source, not just login state.

- Read SOL, USDC and eligible PreStocks balances directly from the connected Solana account.
- Convert SOL ↔ USDC and SOL/USDC → eligible PreStocks through wallet-signed Jupiter routes.
- Move wallet USDC into the Stocklana account with on-chain deposit verification and replay-safe crediting.
- Use the same money for PreStocks, payoff games, collateral, MAQUE payments and Agent Vault budgets.
- Create merchant-bound, one-use purchase intents for ordinary websites.
- Fund an internet purchase from Phantom USDC or existing Stocklana USDC.
- Keep subscriptions blocked by default and bind one-time policies to merchant, amount and expiry.
- Allow scoped agents with `commerce:purchase` authority to buy within their Agent Vault single-spend limit; purchases above the configured human-approval threshold stop for approval.
- Keep wallet private keys in Phantom. Stocklana prepares transactions and policies; the wallet signs locally.
- External ordinary-card checkout uses `CARD\_ISSUER\_PROXY\_URL` / `CARD\_ISSUER\_PROXY\_TOKEN`. Stocklana stores only opaque provider identifiers and optional hosted reveal / wallet-pass handoff URLs, never PAN/CVV.

The PreStocks eligibility lane remains strict: no non-PreStocks pre-IPO mint is admitted by the wallet conversion route.

Run `python3 scripts/test\_wallet\_commerce.py` for the dedicated wallet/commerce regression gate.

Product surface

360-instrument market factory

`src/market-catalog.py` generates **360** unique instruments across 16 families from the eight PreStocks assets:

- valuation thresholds and valuation zones;
- token-price thresholds and price zones;
- premium/discount convergence;
- absolute-move contracts;
- relative-return and valuation-spread pairs;
- joint-positive outcomes;
- basket thresholds and basket leaders;
- universe leaders;
- Gain Games;
- Downside Shields;
- Margin Duels;
- Green-Majority basket games.

All generated templates carry machine-readable settlement rules. `src/settlement-engine.py` automatically evaluates **15** rule families.

Fully collateralized market execution

The production default is a **pari-mutuel collateral pool**, not an unfunded synthetic market maker.

- every YES/NO stake moves into segregated per-market escrow;
- protocol fees move to a separate fee account;
- the house never takes a YES or NO position;
- winning payouts can only come from collateral already present;
- `paidOut` reduces remaining liability after each redemption;
- `src/market\_execution.py` is the shared execution core for humans, agents and tests;
- a market solvency endpoint exposes escrow, collateral, paid-out amount, outstanding liability and house directional exposure.

Embedded PreStocks execution

`src/solana\_finance.py` and `src/solana-client.js` implement Jupiter Swap V2 order/execute paths for buying or selling an eligible PreStocks token through a connected Solana wallet.

A plan is not counted as a fill. A completed external execution requires the provider response and the user's wallet-signed chain result.

Native Phantom wallet workspace

Phantom is now a first-class Stocklana surface instead of an external black box. The connected-wallet workspace reads the user's public Solana state directly and shows:

- SOL and USDC balances;
- eligible PreStocks holdings matched by exact mint;
- Stocklana Vault balance alongside wallet balance;
- direct routes into PreStocks execution, payoff games, collateral/credit, transfers and Agent Vault funding;
- account-change refresh when the user switches Phantom accounts.

Private keys never enter Stocklana. Phantom remains the signing boundary; Stocklana reads public account state, builds transactions and asks the wallet to sign.

On-chain Stocklana market program

`programs/stocklana-market/` contains the current Solana market source:

- exact eight-mint PreStocks allowlist;
- USDC collateral only;
- market / position / vault-authority / fee-authority PDAs;
- SPL-token collateral transfer into program vaults;
- separate fee vault;
- YES/NO pool accounting;
- transferable positions;
- time-gated resolution;
- proportional redemption from escrow;
- fee withdrawal;
- on-chain `resolution\_commitment` binding the Boolean outcome to the external Phantasma/PreStocks/Pyth observation proof.

`src/stocklana-program-client.js` constructs initialize, buy, transfer, resolve, redeem and fee-withdraw transactions using matching PDA derivation.

No Program ID is invented by this package. Deployment requires a Solana build/deployment signer and a confirmed deployment transaction.

Stocklana Vault + PARRALAX clearing lineage

Users and agents operate through the same financial fabric rather than separate wallet silos.

- CASH / TRADING / RESERVE subaccounts;
- internal transfers;
- verified Solana USDC deposits;
- segregated market escrow;
- transferable market positions;
- reserve -> authorize -> chain-confirm withdrawal lifecycle;
- reconstructible public ledger;
- conservation and concurrency testing.

Human authority comes from a one-time Solana wallet challenge. Agent authority comes from scoped capability tokens whose secrets are returned once while only commitments are retained server-side.

100 AI Agent Vaults

`src/agent\_vault.py` provisions policy-scoped Agent Vaults with:

- CASH / TRADING / RESERVE / FEES accounts;
- Solana, EVM, ICP, Bitcoin-watch and Cosmos connector descriptors;
- spend budgets and approval thresholds;
- market permissions;
- reconciliation;
- Sentinel replay / velocity / anomaly checks;
- billing and monetization metadata;
- positions and receipts;
- Phantasma PQ provisioning proof.

The seven-day regression uses ****100 distinct personas**** rather than repeating one scripted actor.

MAQUE Pay Fabric

MAQUE is the payment/account layer underneath the vertical:

- `@handle` transfers;
- QR / NFC / payment-link requests;
- device-bound offline authorization with expiry and nonce replay protection;
- internal settlement;
- Solana payment requests;
- bank/debit/card routing intents;
- human and agent payment identities.

Offline authorization is never mislabeled as offline blockchain finality: the signed intent can be created offline, while settlement finality occurs when a connected rail confirms it.

ISO 20022 interoperability

`src/iso20022\_bridge.py` translates the Stocklana/MAQUE canonical payment object into:

- `pain.001` payment initiation;
- `pacs.008` clearing/settlement;
- `camt.053` account statement/reconciliation;
- `remt.001` remittance context.

This gives Solana, internal clearing, card-processor and bank connectors one shared financial vocabulary.

One-time virtual-card funding controls

`src/card\_rail.py` implements a real internal authorization model:

- one-use spending policy;
- merchant/MCC/amount/expiry controls;
- actual Stocklana funds reserved before approval;
- capture into a card-clearing account;
- cancellation/reversal restoring the reserve;
- hybrid PQ authorization proof;
- opaque external issuer-token handling only-Stocklana never fabricates or stores PAN/CVV.

External issuance requires `CARD\_ISSUER\_PROXY\_URL` and provider authorization.

Lending, simulations and growth

- `src/lending.py`: funded liquidity pool, capped LTV, locked collateral, principal return, interest accounting, no money creation.
- `src/simulations.py`: payoff, basket and liquidity stress scenarios.
- `src/growth.py`: referral attribution, conversion/revenue events and creator-share accounting.

Launch infrastructure

- `contracts/MNTY.sol`: fixed-supply Ethereum token source.
- `scripts/launch/create\_mnty\_solana.mjs`: Solana Token-2022 mint path with optional transfer fee extension and mint-authority revocation.
- `src/meteora-dbc.js`: Meteora DBC config/pool/quote/buy builders with DAMM v2 migration settings.
- `src/clawpump.py`: optional ClawPump partner/self-funded launch adapters.

No launch is represented as complete without the provider/chain receipt.

Phantasma post-quantum finance security

The current high-throughput design is **not** "PQ-sign every tiny ledger event." It is a clearinghouse-style two-level proof system:

1. Every finance event is immediately domain-separated and BLAKE2b-512 hash-linked.
2. High-risk settlement/security events receive an individual **ML-DSA-65 + Ed25519** hybrid signature.
3. Normal event throughput is finalized by a hybrid PQ batch anchor every 16 events.
4. Private journal records use AES-256-GCM under a journal data-encryption key.
5. The journal root key is protected by a hybrid **ML-KEM-768 + X25519** envelope.
6. Runtime private keys are mode 0600 and excluded from release archives.

Laptop + mobile product

The laptop workstation exposes **8** verified views:

1. Market Desk
2. Instrument Forge
3. Vault & MAQUE
4. Positions
5. Agent Network
6. Infrastructure
7. Bounty Matrix
8. Launch

The responsive phone experience remains underneath the workstation layout.

Bounty coverage

`data/bounty-coverage.json` contains at least two implemented/code-backed lanes for every requested category: derivatives, DeFi, tools, games, AI agents, predi

SUBMISSION.md

Stocklana - Solana Tokenized Equities Submission

Submission deadline: Friday, September 25, 2026 at 4:00 PM ET

Wedge: **Stocklana V2** - programmable tokenized-equity ownership that users can immediately buy, automate, collateralize, play, send, spend, delegate to agents and launch around.

Could this be a real app people actually use?

Stocklana starts from a simple problem: tokenized equities can already trade around the clock, but most products still treat them like brokerage holdings with a crypto wrapper.

****Stocklana makes the asset useful after the buy.**** A user can acquire a tokenized equity 24/7, automate recurring investing, combine holdings into baskets, create simple fully collateralized payoff markets, route a supported tokenized share into credit, give an AI agent scoped authority to manage it, transfer value socially, and spend from portfolio value through MAQUE/Solana payment/card rails.

The product is one coherent account fabric rather than a collection of demos.

90-second judge path

1. ****Equity Desk**** - open a live tokenized-equity asset and choose ****Buy / sell 24/7****. Stocklana builds the Jupiter Swap V2 action for the exact SPL mint.
2. Choose ****Schedule recurring buy****. The plan is persisted as a real daily/weekly/monthly investment instruction.
3. ****Instrument Forge**** - turn the same equity into a simple collateralized price/value/relative/basket payoff. Stocklana does not take the other side.
4. ****Vault & MAQUE**** - move funds between CASH/TRADING/RESERVE, send a friend value, generate a Solana payment request, or create a one-use spend policy.
5. ****Positions**** - transfer a market position to another participant.
6. ****Agent Network**** - inspect policy-scoped Agent Vaults. The same account, market and payment code is used by human and AI identities.
7. ****Infrastructure**** - inspect solvency, automatic settlement, Phantasma PQ proofs, ISO 20022 translation and external execution boundaries.
8. ****Track Fit**** - see the exact contest categories mapped to implemented surfaces.

Exact track alignment

Tokenized Equities - round-the-clock assets

PreStocks is the live private-equity universe in this submission. The execution layer is mint-specific and Solana-native. The architecture also isolates a public tokenized-equity adapter (``src/xstocks_adapter.py``) for xStocks/Backed product and price data, so broader Solana equity rails can coexist without contaminating the PreStocks eligibility lane.

DeFi integration

- ``src/kamino_adapter.py`` calls Kamino's transaction API to build unsigned collateral-deposit and borrow transactions. The user's wallet signs locally.
- ``src/lending.py`` supplies a fully funded internal credit rail for Stocklana-native collateral workflows; it cannot create money.

A token is only routed to Kamino when a real Kamino market/reserve supports that asset. Stocklana does not fabricate reserve support.

V2 Solana financial substrate

- ****V2:**** Token-2022 escrow, collateral, credit, agent, position and basket receipts with selective metadata, non-transferability and transfer-hook policy.
- ****Internal accounting:**** eight typed financial claims/capabilities compile from verified events into a double-entry ledger. A dedicated 494-assertion test proves balanced journals, 1:1 monetary backing, funded-credit conservation, agent-budget non-monetary semantics, position/basket lifecycle, and chain-event idempotency.

This is the infrastructure argument for Solana: the same equity account can expose simple SPL compatibility today and more expressive Token-2022 financial semantics where integrations support them.

Programmable assets

- ****360 instruments / 16 families**** derived from eligible tokenized-equity observations.
- ****15 automatic settlement evaluators.****

- recurring buys, weighted baskets, robo allocations, social/copy signals, corporate-action ingestion and AI Agent Vault policy automation.

What the track asks for

Trading

1. Jupiter Swap V2 tokenized-equity ↔ USDC execution.
2. Fully collateralized programmable equity markets with transferable positions.

Investing

1. Daily/weekly/monthly recurring buys.
2. Weighted baskets and policy-scoped robo portfolios.

Credit and yield

1. Kamino unsigned transaction construction for supported equity collateral reserves.
2. Pool-funded Stocklana credit plus corporate-action/dividend event ingestion.

Infrastructure

1. PreStocks observation + Pyth where a relevant feed exists, automatic settlement and per-market solvency analytics.
2. Corporate-action registry, wallet-authenticated identities, Phantasma PQ receipts, reconstructible ledger and ISO 20022 reconciliation.

Consumer

1. Laptop/mobile equity workstation plus social/copyable thesis signals.
2. MAQUE internal payments, Solana payment requests and one-time spend controls sourced from portfolio value.

Why this belongs on Solana

This product depends on the equity being a programmable token, not a database row at a brokerage. On Solana the same SPL/Token-2022 asset can move through wallet execution, program escrow, payments, credit and agent policy. Fast low-cost settlement lets investing, DeFi and payments share one account model.

End-to-end proof

- **360** instruments across **16** families.
- **15** automatic settlement rule families.
- exact eight-mint PreStocks private-equity allowlist.
- fully collateralized pari-mutuel default and segregated market escrow.
- house directional exposure = **0** on exercised markets.
- transferable positions.
- wallet-authenticated humans + scoped AI capabilities.
- **100** Agent Vaults.
- **100** personas × 7 days = 700/700 sessions, 0 failures**.
- **1,343** acceptance assertions PASS** plus **494** accounting-token assertions PASS** after track-specific additions.
- **8/8** workstation navigation surfaces.
- **100/100** ISO 20022 translation stress check.
- hybrid Phantasma PQ proof architecture.

See `TRACK-FIT.md`, `ACCEPTANCE-RESULT-V7.json`, `WEEK-100-REPORT.md`, `REQUIREMENTS-AUDIT.md`, `SECURITY.md` and `public/ledger.csv`.

On-chain execution boundary

`programs/stocklana-market/` implements the current USDC escrow market source and `src/stocklana-program-client.js` contains matching transaction builders: PreStocks allowlist, PDAs, USDC collateral vault, fee vault, YES/NO pools, transferable positions, time-gated resolution, proof commitment and proportional redemption.

This archive does **not** invent a Program ID or transaction signature. A live Solana deployment requires a funded deployment signer and confirmed chain transaction. The same rule applies to external Jupiter fills, Kamino deposits/borrows, Meteora pools, card issuance and bank settlement: the code path exists, but the external provider/chain receipt is the execution proof.

SECURITY.md

Stocklana v6 Security Architecture

Design rule

Cryptography, authorization, accounting consistency and external settlement verification are independent controls. None substitutes for the others.

Phantasma PQ suite

Immediate event integrity

Every finance event receives a domain-separated **BLAKE2b-512** commitment and is linked to the previous receipt head.

High-risk finality

Settlement-sensitive/high-risk events receive individual hybrid signatures:

- **ML-DSA-65** post-quantum signature;
- **Ed25519** classical compatibility signature.

Throughput finality

Routine events are not forced through a heavyweight PQ signer one-by-one. They remain immediately hash-linked and are finalized in **hybrid PQ batch anchors every 16 events**. This preserves high-throughput payments while retaining post-quantum batch authenticity.

Private journal

- AES-256-GCM encrypts private journal records.
- A journal root DEK protects the record stream.
- The root key is wrapped through **ML-KEM-768 + X25519** hybrid key establishment.
- runtime private keys are mode 0600 and excluded from release archives.

Financial authorization

Humans

A human session starts from a one-time Solana wallet challenge. The wallet signs the challenge and the server verifies the Ed25519 signature. An arbitrary request-body `trader`` value has no production authority.

Agents

AI actors receive scoped capability tokens. The secret is returned once; only the BLAKE2b commitment is stored. Vault policy controls budgets, approvals and action scope.

Market accounting

The default market model is fully collateralized pari-mutuel:

1. user funds are debited from an authorized vault;
2. stake enters `stocklana:market:<id>:escrow``;
3. protocol fee moves separately to the fee account;
4. position shares represent stake in YES or NO pool;
5. resolution selects the winning pool;

6. redemption transfers only from existing market escrow;
7. `paidOut` reduces outstanding liability;
8. solvency requires escrow $\geq$ outstanding liability;
9. Stocklana never owns the YES or NO side.

All balance/position mutations use serialized single-writer locking in the local clearing engine.

External Solana funding verification

A chain-backed credit/trade can only be accepted after RPC evidence proves:

- transaction succeeded;
- expected USDC mint was used;
- configured Stocklana vault received sufficient USDC;
- claimed sender funded the transaction as expected;
- signature has not already been credited.

External signatures are one-use settlement references.

Withdrawal state machine

`REQUEST -> RESERVED -> AUTHORIZED -> CHAIN TRANSFER -> RPC VERIFIED -> SETTLED`

Cancellation releases the reserve. A withdrawal is not marked settled merely because an internal balance was debited.

Automatic resolution proof

Keeper settlement stores the source, observation timestamp, exact rule inputs, outcome and signed proof commitment. The Solana program source stores a 32-byte resolution commitment with the resolved outcome so the chain state can be tied to the external proof package.

MAQUE offline security

Offline payment packets are device-bound Ed25519 authorizations with:

- expiry;
- one-time nonce;
- replay rejection;
- later Phantasma receipt finality when accepted by Stocklana.

Offline authorization is not described as offline chain settlement.

One-time card controls

Stocklana reserves the authorized funds before a one-use card policy can be approved. Capture moves reserved funds into card clearing; cancellation restores them. External PAN/token issuance is delegated to an authorized issuer adapter, and Stocklana stores only opaque provider references-not PAN/CVV.

Lending conservation

Credit cannot mint balance. Principal comes from `stocklana:lending-pool`; collateral is locked before draw; principal returns to the pool; interest is accounted separately; collateral releases only after repayment conditions pass.

Deployment secrets never shipped

Do not package:

- PQ private PEM keys;
- Solana treasury/deployer private keys;
- Jupiter/Pyth/ClawPump/card-provider credentials;
- raw card PAN/CVV;
- bank credentials.

Public configuration and verification keys may ship; signing authority does not.

DEPLOY-SOLANA.md

Stocklana - Hybrid Solana Release

Stocklana ships as two production layers. This is intentional.

1. Solana executable layer

For judging, Stocklana now defaults its custom coordination program to **Solana Devnet** while the real PreStocks/Jupiter wallet lane can remain on Solana mainnet-beta. This is the intended hybrid architecture: decentralized trust / market state on Solana, heavier orchestration and external-provider work off-chain.

The on-chain Stocklana market program lives in `programs/stocklana-market`. It enforces:

- the exact eight PreStocks mints;
- USDC collateral;
- fully collateralized YES/NO pools;
- market / position / vault / fee PDAs;
- transferable positions;
- resolution commitments and redemption.

A Devnet judging deployment does not require paid SOL; the deployment script requests faucet SOL when the signer is low. Never commit a signer keypair.

```
```bash
export SOLANA_KEYPAIR_PATH="$HOME/.config/solana/id.json"
npm run deploy:program:devnet
```
```

Mainnet remains available later:

```
```bash
npm run deploy:program:mainnet
```
```

The script builds with `cargo build-sbf`, requires an explicit cluster confirmation, submits the program, and writes the Program ID plus CLI receipt under `deployments/`.

After deployment set:

```
```bash
export STOCKLANA_PROGRAM_CLUSTER='devnet'
export STOCKLANA_PROGRAM_RPC_URL='https://api.devnet.solana.com'
export STOCKLANA_PROGRAM_ID='<confirmed program id>'
```
```

The user-facing PreStocks wallet rail remains separately configurable with `SOLANA\_RPC\_URL` and defaults to mainnet-beta.

Do not call the program deployed until `solana program show <PROGRAM\_ID>` confirms it.

2. Permanent browser layer controlled by a Solana wallet

The browser app cannot literally execute from inside a Solana program. The production web release is built as static files, then can be stored permanently on Arweave through Turbo using a Solana deployment wallet.

Install dependencies:

```
```bash
npm install
```
```

Build a permanent-web bundle:

```
```bash
export STOCKLANA_PUBLIC_API_ORIGIN='https://api.stocklana.example'
npm run build:static
```
```

Upload the bundle with the Solana wallet:

```
```bash
export SOLANA_KEYPAIR_PATH="$HOME/.config/solana/id.json"
npm run deploy:frontend:solana
```
```

The deployer writes a second receipt under `deployments/` with the immutable Arweave manifest ID and permanent gateway URL.

The API origin is intentionally runtime-configurable so the permanent frontend can call Stocklana's off-chain orchestration/API service while Phantom continues signing Solana actions locally.

For a stable human-readable permanent name, bind the resulting manifest to an ArNS name controlled by the Solana owner after the first successful permanent upload.

3. Production API

If the permanent frontend is not served from the same origin as `server.py`, configure the exact frontend origin:

```
```bash
export STOCKLANA_CORS_ORIGIN='https://<your-permanent-or-custom-domain>'
python3 server.py --host 0.0.0.0 --port 5173
```
```

Keep these secrets only in the API deployment environment:

- `JUPITER\_API\_KEY`
- `PYTH\_API\_KEY`
- `CARD\_ISSUER\_PROXY\_TOKEN`
- `CLAWPUMP\_API\_KEY`
- any infrastructure provider secrets

4. CMESH is a separate chain lane

CMESH / CipherMesh is ****not a PreStock****.

It remains:

- role: Stocklana platform token
- origin: Pons
- chain: Robinhood Chain
- chain ID: 4663

Set the exact verified contract after confirmation:

```
```bash
export STOCKLANA_CMESH_ADDRESS='0x...full verified CMESH token address...'
```
```

The CMESH tab verifies the address against the Pons factory before showing it as CMESH. Do not place the token in `data/prestocks-snapshot.json`, the Solana PreStocks allowlist, or any PreStocks bounty route.

Receipt rule

A release is complete only when the corresponding external receipt exists:

- Solana market program: Program ID + deployment signature / `solana program show`
- permanent frontend: Arweave manifest transaction ID
- CMESH: exact token address + Robinhood Chain/Pons state
- swaps/trades/purchases: provider or chain receipt

Never substitute a configured route, local test, or transaction builder for a mainnet receipt.

TRACK-FIT.md

Stocklana - Tokenized Equities Track Fit

The wedge

****Stocklana makes a tokenized equity useful after you buy it.****

A brokerage app mostly lets a user buy, hold and sell. Stocklana treats a tokenized equity as a programmable Solana asset that can be acquired 24/7, scheduled, combined into baskets, governed by an AI agent, used in a fully collateralized market, routed into supported credit venues, transferred socially, and used as the source of portfolio spending.

The product is deliberately one wedge: ****programmable tokenized-equity ownership****. The 360-instrument market factory, Stocklana Vault, Agent Vaults, MAQUE and Phantasma are infrastructure behind that wedge, not separate pitches.

What the source asks for → what the build does

Trading

The private-equity lane uses PreStocks; the public-equity adapter is isolated for xStocks/Backed data and issuer-specific rules.

1. ****24/7 tokenized-equity swaps**** - the PreStock asset action builds Jupiter Swap V2 orders for the exact SPL mint and supports wallet-signed execution. USDC is the settlement asset.
2. ****Programmable equity markets**** - 360 fully collateralized instruments let users express simple price, valuation, relative-value, basket and downside views without Stocklana taking the opposite side.

Investing

1. **Recurring buys** - `src/equity\_os.py` persists daily/weekly/monthly recurring-buy plans and emits the next Jupiter execution action.
2. **Index baskets + robo portfolios** - weighted multi-equity baskets compile into per-mint Jupiter orders; policy-scoped robo allocations support conservative/balanced/growth portfolios.

Credit and yield

1. **Tokenized-equity collateral** - `src/kamino\_adapter.py` builds real Kamino unsigned deposit/borrow transactions through `api.kamino.finance`; the user's wallet signs locally. A stock can only be routed when a live Kamino reserve for that mint exists.
2. **Credit + corporate action layer** - Stocklana's funded lending pool never creates money; the equity OS also records issuer/provider corporate actions so dividends, splits or similar events can feed automation and accounting rather than being ignored.

Infrastructure

1. **Observation and settlement** - PreStocks observations feed the private-equity lane; Pyth is used where a relevant Pyth feed exists; automatic settlement never uses the fallback UI snapshot for money-bearing resolution.
2. **V1/V2 financial substrate** - a double-entry financial-token grammar compiles deposits, escrow, collateral, fees, funded credit, agent budgets, positions and baskets into typed claims. V1 uses classic SPL receipts for compatibility; V2 uses Token-2022 non-transferability / transfer hooks where policy should travel with the asset.
3. **Operations** - wallet-authenticated sessions, per-market solvency, corporate-action registry, Phantasma PQ receipts, reconstructible ledger and ISO 20022 translation.

Consumer

1. **Investing interface** - responsive mobile plus laptop workstation, human-readable asset actions, recurring plans, baskets and social/copy signals.
2. **Spend from portfolio** - MAQUE internal payments, Solana payment requests and one-use card policies make portfolio value usable without forcing the user through a brokerage-style sell/withdraw/redeposit workflow.

Why Solana

This product only makes sense when the equity is a programmable token. Stocklana deliberately ships **both** Solana modes: V1 classic SPL receipts maximize compatibility with the existing ecosystem, while V2 Token-2022 receipts add policy semantics such as non-transferability and transfer hooks to escrow, collateral, agent, position and basket claims. The same account can therefore use today's DeFi rails without giving up a path to richer programmable-asset controls.

Track proof gate

`python3 scripts/test\_track\_fit.py` verifies the source-aligned wedge, at least two implemented surfaces in every judging category, recurring buys, basket compilation, robo allocation, corporate actions and the Kamino transaction-builder boundary. `python3 scripts/test\_accounting\_tokens.py` independently runs 494 assertions over the V1/V2 financial substrate.

REQUIREMENTS-AUDIT.md

Stocklana v6 - Last-Three-Directives Acceptance Audit

This document is the release gate. A feature counts only when it has code, an execution path, and proof. External-network actions are never reported as executed without provider/network receipts.

Summary

- Requirements audited: **44**
- Executed + proven in-system: **33**
- Fully coded; external credential/wallet/provider authorization required: **7**
- Source/client ready; actual chain deployment still requires a deployment signer/toolchain: **4**
- Market catalog: **360** instruments / **16** families
- Automatic evaluator: **15** rule families
- Week run: **100** personalities / **700** sessions / **0** failures
- Acceptance suite: **1,287** assertions

Requirement ledger

| ID | Requirement | Status | Primary evidence |
|-----|--|----------------------------------|---|
| --- | --- | --- | --- |
| R01 | Laptop trading workstation | EXECUTED_PROVEN | index.html, src/styles.css |
| R02 | Responsive phone experience preserved | EXECUTED_PROVEN | index.html, src/styles.css |
| R03 | Every navigation tab resolves to a real view | EXECUTED_PROVEN | index.html, src/app.js |
| R04 | PreStocks-only eligibility for private-company underlyings | EXECUTED_PROVEN | src/market-store.py, programs/stocklana-market/src/lib.rs |
| R05 | Large reinvented instrument universe | EXECUTED_PROVEN | src/market-catalog.py |
| R06 | Automatic settlement for every market family | EXECUTED_PROVEN | src/settlement-engine.py, src/keeper.py |
| R07 | No Stocklana directional side / fully collateralized default | EXECUTED_PROVEN | src/market-store.py, src/market_execution.py |
| R08 | Segregated per-market escrow and fee account | EXECUTED_PROVEN | src/market_execution.py, src/finance-store.py |
| R09 | Transferable YES/NO positions | EXECUTED_PROVEN | src/market-store.py, src/market_execution.py |
| R10 | Embedded direct access to real PreStocks token market | CODED_EXTERNAL_AUTH_REQUIRED | src/solana_finance.py, src/solana-client.js |
| R11 | Solana USDC chain-backed market funding verification | CODED_EXTERNAL_AUTH_REQUIRED | server.py: verify_usdc_deposit + trade-chain |
| R12 | Deployable on-chain Stocklana prediction market | SOURCE_READY_DEPLOYMENT_REQUIRED | programs/stocklana-market/src/lib.rs, src/stocklana-program-client.js |
| R13 | Resolution tied to Phantasma observation proof | SOURCE_READY_DEPLOYMENT_REQUIRED | src/keeper.py, programs/stocklana-market/src/lib.rs |
| R14 | Pyth market-data/oracle adapter | CODED_EXTERNAL_AUTH_REQUIRED | src/settlement-engine.py |
| R15 | CASH/TRADING/RESERVE vault | EXECUTED_PROVEN | src/finance-store.py, index.html |
| R16 | Internal user-to-user transfers | EXECUTED_PROVEN | src/finance-store.py, server.py |
| R17 | Withdrawal reserve -> chain proof -> settlement lifecycle | EXECUTED_PROVEN | src/finance-store.py, server.py |
| R18 | MAQUE @handle payments | EXECUTED_PROVEN | src/payment_fabric.py, server.py |
| R19 | QR/NFC/payment request transport | EXECUTED_PROVEN | src/payment_fabric.py |
| R20 | Offline payments with replay protection | EXECUTED_PROVEN | src/payment_fabric.py |
| R21 | Solana Pay merchant/payment requests | EXECUTED_PROVEN | src/solana_finance.py, server.py |
| R22 | ISO 20022 banking-language bridge | EXECUTED_PROVEN | src/iso20022_bridge.py |
| R23 | Debit/bank/on-ramp routing | CODED_EXTERNAL_AUTH_REQUIRED | server.py: api/funding/onramp, src/payment_fabric.py |
| R24 | One-time virtual card controls backed by reserved funds | EXECUTED_PROVEN | src/card_rail.py, src/finance-store.py |
| R25 | Actual external virtual-card issuance connector | CODED_EXTERNAL_AUTH_REQUIRED | src/card_rail.py: issue_virtual |
| R26 | 100 AI Agent Vaults | EXECUTED_PROVEN | src/agent_vault.py |
| R27 | Scoped AI financial capabilities | EXECUTED_PROVEN | src/auth.py, src/agent_vault.py |
| R28 | Wallet-signed human financial sessions | EXECUTED_PROVEN | src/auth.py, src/app.js |
| R29 | Phantasma post-quantum security | EXECUTED_PROVEN | src/pq_crypto.py, src/finance-store.py |
| R30 | High-throughput PQ finance receipts | EXECUTED_PROVEN | src/finance-store.py |
| R31 | Lending/collateral without money creation | EXECUTED_PROVEN | src/lending.py |
| R32 | Structured products and baskets | EXECUTED_PROVEN | src/market-catalog.py |
| R33 | Payoff/liquidity/basket simulation tools | EXECUTED_PROVEN | src/simulations.py |
| R34 | Growth/referral monetization experiments | EXECUTED_PROVEN | src/growth.py |
| R35 | At least two surfaces in every bounty-requested category | EXECUTED_PROVEN | data/bounty-coverage.json, index.html |
| R36 | Meteora Dynamic Bonding Curve integration | CODED_EXTERNAL_AUTH_REQUIRED | src/meteora-dbc.js |
| R37 | Optional Clawpump bounty compatibility | CODED_EXTERNAL_AUTH_REQUIRED | src/clawpump.py, server.py |
| R38 | Native Stocklana launch path and fee model | SOURCE_READY_DEPLOYMENT_REQUIRED | contracts/MNTY.sol, scripts/launch/create_mnty_solana.mjs |
| R39 | MNTY on Ethereum and Solana | SOURCE_READY_DEPLOYMENT_REQUIRED | contracts/MNTY.sol, scripts/launch/create_mnty_solana.mjs |
| R40 | 100 personalities run for seven days | EXECUTED_PROVEN | scripts/week100_checkpoint.py, data/agent-personas.json |
| R41 | Judge-grade proofroom and reconstructible ledger | EXECUTED_PROVEN | SUBMISSION.md, JOURNAL.md |
| R42 | Software company extends beyond stock trading | EXECUTED_PROVEN | MAQUE.md, src/payment_fabric.py |

| R43 | AI-native by default | `EXECUTED\_PROVEN` | src/agent_vault.py, src/auth.py |
| R44 | Monetization without taking market direction | `EXECUTED\_PROVEN` | src/market-store.py, src/growth.py |

External execution gates that remain real gates

These are not product placeholders. Their transaction builders/adapters exist, but Stocklana will not invent a receipt:

- **Stocklana Solana program deployment:** requires compiled SBF plus deployment signer; no Program ID is claimed until Solana confirms it.
- **Jupiter Swap V2 fills:** requires Jupiter API access and the user wallet signature; an order plan is not counted as a fill.
- **Meteora DBC:** config/pool/swap builders are implemented; mainnet creation requires the user wallet/RPC and the relevant quote-mint compatibility.
- **ClawPump:** partner/self-funded launch adapters exist; execution requires a `cpk\_` partner credential and payment proof.
- **Virtual card issuance:** Stocklana reserves funds and creates one-use policy; actual card issuance requires the issuer/processor connector.
- **Bank/debit on-ramp/off-ramp:** routing and ISO 20022 semantics exist; settlement requires provider confirmation.
- **MNTY:** fixed-supply Ethereum source and Solana Token-2022 mint/revoke path exist; no mainnet contract/mint address is claimed before the signing transaction.

Non-negotiable accounting/security invariants

- Stocklana market default is fully collateralized pari-mutuel; the house does not take YES or NO.
- Market collateral moves to a segregated market escrow; fees move separately.
- Redemptions decrease outstanding liability via `paidOut`; solvency compares escrow with remaining liability.
- Human account authority comes from Solana wallet challenge signatures; arbitrary `trader` strings are disabled outside explicit dev mode.
- Agent authority is scoped capability tokens; only token commitments are stored.
- Finance events are immediately BLAKE2b-512 hash-linked. High-risk events receive individual ML-DSA-65 + Ed25519 signatures. Normal throughput is finalized by PQ batch anchors every 16 events.
- Private journal records use AES-256-GCM under a journal DEK that is itself protected by ML-KEM-768 + X25519.

Proof artifacts

- `WEEK-100-PROOF.json` / `WEEK-100-REPORT.md`
- `data/requirements-ledger.json`
- `data/bounty-coverage.json`
- `scripts/test\_acceptance.py`
- `SUBMISSION.md`, `JOURNAL.md`, `LEDGER.md`, `SECURITY.md`, `public/ledger.csv`

DESIGN-LOCK.md

STOCKLANA DESIGN LOCK

Status: canonical visual contract
Locked: 2026-09-25
Product source of truth: this repository

The user-approved Stocklana dashboard image is the visual source of truth for the product shell. Do not reinterpret it into a generic crypto dashboard and do not rebuild Stocklana as a parallel project.

Non-negotiable visual contract

The desktop composition is:

1. fixed black/navy left rail;

2. Stocklana mark and brand at the top;
3. Home, PreStocks, Convert, Play, Shop, AI Agent, Send, Portfolio;
4. bottom rail promotional visual, which also opens the separate CMESH platform-token surface;
5. top-center command/search bar;
6. top-right activity + Connect Phantom;
7. large left hero headline:
 - Your money shouldn't
 - ****stop working**** after
 - you invest it.
8. approved neon planet/orbit media on the right;
9. six compact action cards;
10. two-row dashboard:
 - Wallet
 - Featured PreStocks
 - Play / prediction payoff
 - Shop
 - AI budget
 - Activity

Visual material: near-black field, cyan edge light, acid-lime primary accent, violet secondary accent, subtle glass, 13-19px radii, dense but calm spacing.

Canonical assets

- `assets/hero-reference.webp` - approved hero-media crop from the design lock.
- `assets/shop-products-clean.webp` - approved shop-product artwork crop.
- The rest of the experience stays live HTML/CSS/JS. Never replace the full app with a screenshot.

Product integrity

Visual work must not delete or bypass:

- Phantom wallet ownership/signing boundary;
- PreStocks exact-mint eligibility;
- Jupiter routes;
- Stocklana market / settlement engines;
- Play and the Solana judging loop;
- Money / MAQUE;
- Shop / one-time commerce controls;
- Agent Vault policies;
- Credit;
- Launch;
- Portfolio;
- System / Proof;
- CMESH.

CMESH is ****not** a PreStock**. It stays in the separate Pons / Robinhood Chain platform-token lane.

Internal pages

Every internal page must use the same design grammar as Home. It may have its own product-world visual, but it must not regress into generic white-label panels.

The locked shell and internal-page design rules live at the bottom of `src/styles.css` under:

- `ONE-TO-ONE MOCKUP LOCK V3`

- `INTERNAL ONE-TO-ONE EXPERIENCE LOCK`

The internal product-world renderer is `src/surface-motion.js`.

Safety / rollback

Do not delete these branches:

- `backup/pre-one-to-one-design-lock-2026-09-25`
- `checkpoint/one-to-one-home-lock-2026-09-25`

They preserve the product before and at the one-to-one visual checkpoint.

Design changes are targeted patches to the canonical app. Never create V14/V15/side-app replacements unless explicitly requested.

Verification rule

A page is not visually certified because the CSS exists. Compare a real browser render to the approved design reference at the target desktop viewport before calling the pass complete.

External financial or chain actions are not complete without their real provider / chain receipt.